RICE UNIVERSITY

T2 · Distributed programming

Write a CPU ring all-reduce with mpi4py, benchmark NCCL collectives with torch.distributed, and fit latency and bandwidth.

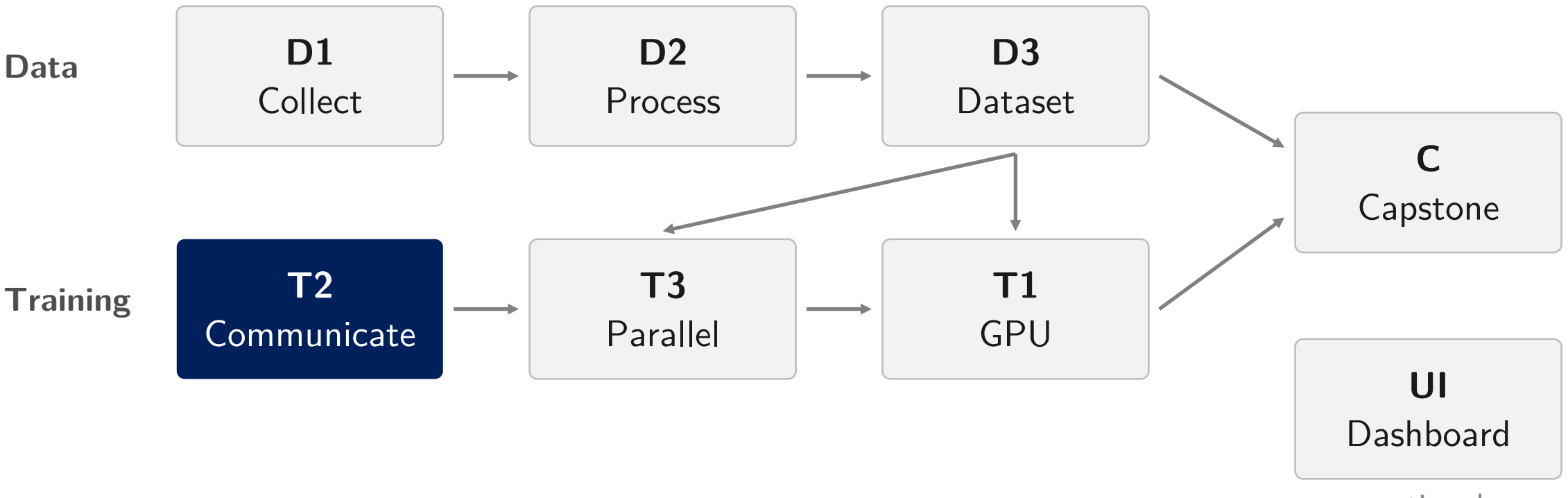
COMP 329 course staff

Rice University

Spring 2027 · Assignment T2

Release week 7 · Checkpoint week 9 · Due week 10 · 7% of grade

Where this assignment fits



In our pipeline

This assignment: `no dataset` → **T2** → `assignment-result.json`

What you will build, and why

Make p processes add up their arrays: first with your own messages, then with NCCL, and predict the time with $\alpha + n \cdot \beta$.

1 **Training needs sums**

Data parallelism all-reduces gradients every step.

2 **Latency vs. bandwidth**

Small messages pay α; large ones pay $n \cdot \beta$. The α–β model predicts both.

3 **Every rank must agree**

Mismatched collectives hang; rank order matters for gathers.

What goes in, what comes out



Files you produce

`mpi_ring`
Your ring vs. MPI Allreduce, with α–β fits

`torch_collectives`
All-reduce, reduce-scatter, all-gather timings

`rank_map`
Which rank ran on which host and GPU

A ring all-reduce in two phases

1

Split

p chunks; lengths may differ by one.

2

Reduce-scatter

$p - 1$ steps: send right, add what arrives.

3

Own one chunk

Rank r holds the full sum of chunk $r + 1$.

4

All-gather

$p - 1$ steps: pass finished chunks on.

5

Done

$2(p - 1)$ messages of n/p per rank.

Key idea

$T(n, p) \approx 2(p - 1) \cdot \alpha + 2(p - 1)/p \cdot n \cdot \beta$. Trace 3 ranks by hand before coding.

Three collectives

All-reduce

`send vector → get the sum`

Bus factor $2(p-1)/p$.

Reduce-scatter

`send vector → get one slice of the sum`

Bus factor $(p-1)/p$.

All-gather

`send one slice → get all slices`

Slot i must hold rank i's piece.

Key idea

Check values first (rank r contributes $r+1$), then time: barrier, synchronize, run, synchronize, MAX over ranks.

Fit and use the α–β model

```
A = np.column_stack(
    [np.ones(n), nbytes])
(alpha, beta), *_ = \
    np.linalg.lstsq(A, seconds)
bandwidth = 1 / beta
```

1. α

   Fixed cost per message: dominates below ~1 MiB.

2. $1/\beta$

   Bandwidth: dominates for large messages.

3. r^2

   Report it; a negative β means an unreliable fit.

4. **Predict, then compare**

   Explain the gap with synchronization or topology.

What you write, what you get

You write

`ring_allreduce`

`starter/collectives.py`

`benchmark_collective`

`starter/collectives.py`

`fit_alpha_beta`

`starter/collectives.py`

Provided

- Send/receive and built-in collectives
- Launcher and rank → GPU mapping
- In-process test communicator
- Guided two-server commands

Ground rules

Keep the signatures · don't edit `starter/pipeline_lab` · run through `course.py` or `lab.py`.

Build it in this order

1	2	3	4	5
Trace a ring	**Ring all-reduce**	**Benchmark**	**Fit**	**Measure**
3 ranks, by hand; write the formula.	Sendrecv only; input unchanged.	Check, then time; MAX over ranks.	Least squares α and β.	Size, rank count, topology.
`check t2 --checkpoint`	`check t2 --checkpoint --systems`	`check t2 --systems`	`lab.py t2`	

Check

Done when: All 11 tests pass with --systems (8 checkpoint tests); predictions are compared with measurements.

If something goes wrong

Wrong sum in one chunk
Check: Chunk numbers mod p in each phase.

Timeout
Check: Same steps and matching tags on every rank.

Message-count test fails
Check: One chunk per message, only to the right.

All-gather check fails
Check: Rank order: slot i is rank i.

Microseconds for 16 MiB
Check: Synchronize the GPU before stopping the clock.

Negative β
Check: Add larger message sizes.

The performance study

What limits a collective: latency, bandwidth, or synchronization?

Predict first

Ring: $2(p-1)\cdot\alpha + 2(p-1)/p \cdot n\cdot\beta$. Predict each size from your fit before measuring the next setting.

Message size
1 KiB … 16 MiB

Rank count
2 / 3 / 4

Topology
CPU ranks, GPUs, two servers

How to measure: Two warmups, then repeats; report medians and the slowest rank. Compare predicted and measured time.

How you are graded

20	40	16	24
Checkpoint	Final correctness	Performance	Analysis

Correctness: ring all-reduce 14 · collective checks and rank order 12 · timing and bandwidth 8 · α–β fit 6

Check

Done when: All 11 tests pass with --systems (8 checkpoint tests); predictions are compared with measurements.

Your first ten minutes

```
$ bash setup.sh
$ source .venv/bin/activate
$ python course.py preflight
$ python course.py check t2 --checkpoint
$ python course.py check t2 --systems
$ python lab.py t2 --output runs/t2-first \
    --ranks 2 4 --world-sizes 2
```

Your folder

`05_t2_triton/`

Handout:
`05_t2_triton/README.md`